

BO2-02

Playbooks de nettoyage versionnés

Rapport de réalisation - 8 octobre 2026
Branche de travail : ahmed-ikbel

1. Objectif du DSO

Dans BO2 - Standardize & Automate Data Governance, ce DSO vise à éviter que des problèmes de qualité similaires soient traités différemment selon les projets. Un playbook décrit des règles explicites, rattachées à une version, au contexte du dataset et au rôle des colonnes.

Le prototype sélectionne une politique JSON, valide son applicabilité, exécute des transformations déterministes et produit un journal des corrections et des exceptions. Les décisions ambiguës restent à réviser.

2. Réalisation livrée

Composant	Réalisation
Données	500 clients synthétiques créés directement avec des formats et valeurs imparfaits, dans les 7 colonnes du projet.
Politiques	Trois versions JSON conservées : 1.0.0, 1.1.0 et 1.2.0.
Exécution	Moteur Python/pandas séparé de la présentation Streamlit.
Interface	Choix de version, avant/après, corrections, exceptions et exports CSV/JSON.
Validation	6 tests automatisés réussis, incluant le parcours Streamlit.

Périmètre

Ce rapport couvre BO2-02. La détection contextuelle des anomalies (BO2-07), le masquage PII (BO2-03) et le graphe de traçabilité par colonne (BO2-05) ne sont pas réalisés dans cette étape. Le prototype BO3 existant est conservé.

Données de test et génération

3. Même schéma, nouveau jeu de données

Colonne	Rôle dans ce test
age	Âge numérique entier ; intervalle accepté : 0 à 120.
income	Revenu ; valeur numérique non négative.
spending_score	Score numérique compris entre 0 et 100.
transaction_amount	Montant numérique non négatif dans le contexte de démonstration.
num_transactions	Nombre entier de transactions, non négatif.
true_anomaly	Annotation de simulation fixée à False dans ce générateur.
anomaly_type	Annotation de simulation fixée à none dans ce générateur.

4. Défauts présents dès la création

Le générateur construit chaque enregistrement champ par champ. Pour chaque champ numérique, il tire un mode de génération : valeur normale, nombre au format texte, valeur absente, champ vide, texte inattendu, valeur négative, violation de domaine, unité, séparateurs ambigus ou marqueur non fini. Il ne part pas d'un DataFrame propre à modifier ensuite.

La probabilité de génération normale est de 76 % par champ ; les autres modes représentent 24 % au total. Les défauts sont répartis sur les lignes et plusieurs peuvent concerner une même ligne. Une graine fixe (2026) rend le jeu reproductible.

Les deux annotations de simulation ne sont pas des indicateurs de qualité. False et none signifient ici qu'aucune anomalie statistique étiquetée n'est créée, même si la ligne contient un problème de nettoyage.

Fichiers produits

data/customers_dirty.csv : données brutes, 500 lignes et 7 colonnes.

data/customers_dirty_defects.csv : catalogue séparé des modes de génération et traitements attendus. Il sert à vérifier le test, sans ajouter de colonnes au dataset ni piloter le nettoyage.

Politiques et exécution

5. Évolution des politiques

Version	Comportement
1.0.0	Signale les valeurs numériques absentes. Sur colonnes optionnelles : catégorie Unknown, date ISO et doublons d'identifiants à réviser.
1.1.0	Ajoute le format de date jour/mois/année. Ces colonnes optionnelles sont absentes du dataset courant.
1.2.0	Applique la normalisation numérique aux cinq colonnes métier et vérifie leurs contraintes. Version adaptée au nouveau dataset.

6. Corriger seulement ce qui est défini

La politique 1.2.0 retire les espaces autour d'un nombre et interprète la virgule comme séparateur décimal. Par exemple, le texte « 42,50 » devient le nombre 42.5. Dans ce contexte de démonstration, la virgule n'est jamais un séparateur de milliers.

Les valeurs absentes, les textes non numériques, les unités, les séparateurs ambigus, les valeurs non finies et les violations des contraintes sont signalés pour révision. Un revenu manquant n'est pas remplacé par une médiane ; un âge négatif n'est pas deviné. Les valeurs extrêmes compatibles avec les contraintes ne sont pas corrigées simplement parce qu'elles sont rares.

7. Responsabilités et traçabilité

DQ : choix de la politique applicable au contexte.

SEM : validation des rôles et contraintes déclarés.

ETL : exécution des règles déterministes sur une copie des données.

Ces responsabilités sont représentées par des fonctions et métadonnées locales, et non par trois agents autonomes. Le statut `approved_demo` concerne uniquement la démonstration ; il ne constitue pas une approbation métier de production.

Chaque événement contient la position de ligne (à partir de zéro), la colonne, l'identifiant de règle, l'action, les valeurs avant/après et la raison. Le manifeste conserve la version, une copie de la politique et les empreintes SHA-256 de la politique et des données avant/après. Les colonnes optionnelles absentes sont signalées comme non applicables.

Résultats et utilisation

8. Résultats vérifiés

Sur les 500 lignes générées avec la graine 2026, la politique 1.2.0 donne les résultats suivants. Une ligne peut comporter plusieurs exceptions : le nombre de cellules et le nombre de lignes ne sont donc pas interchangeables.

Mesure	Résultat
Cellules imparfaites à la génération	617
Cellules corrigées automatiquement	196
Exceptions au niveau cellule	421
Lignes nécessitant une révision	304
Lignes conservées après traitement	500

Les 6 tests couvrent : données originales inchangées ; comparaison des anciennes versions et déterminisme ; rejet des contextes/rôles/schémas incompatibles ; index non uniques et entrée vide ; parcours Streamlit ; génération reproductible avec schéma identique et traitement conforme au catalogue. Une seconde exécution sur les données traitées ne produit plus de nouvelles corrections.

9. Démonstration dans Streamlit

1. Lancer depuis le projet : `.venv\Scripts\python -m streamlit run pages/1_BO2_Playbooks.py`.
2. Choisir **1.2.0** et le nouveau dataset avec défauts.
3. Consulter les données brutes, puis cliquer sur **Appliquer le playbook**.
4. Examiner les compteurs, les corrections avant/après et les lignes à réviser.
5. Exporter le CSV et le journal JSON. Le CSV contient encore les exceptions non résolues.

10. Limites et suites

Les résultats sont conservés dans la session Streamlit ; il faut exporter le journal pour garder une preuve. Il n'existe pas encore de workflow de validation humaine, de stockage persistant des exécutions ou de transfert automatique vers les agents aval. Les règles métier (par exemple montants non négatifs) doivent être approuvées pour chaque contexte réel.

Les prochaines étapes possibles sont la validation métier des politiques, un historique persistant des exécutions, puis l'intégration de BO2-07 pour analyser les observations inhabituelles sans les assimiler automatiquement à des erreurs.

Sources internes : `data/dirty_generator.py` ; `governance/playbooks.py` ; `politiques/customers-*.json` ; `pages/1_BO2_Playbooks.py` ; `tests/test_playbooks.py`. État du prototype au 8 octobre 2026.